



SENTINEL-X

L'avant-poste industriel du futur — dossier d'ingénierie technique

Consortium FORTEX — Groupe 20

Prototype cyber-physique : boîtier de surveillance autonome et PC serveur local

Variante d'architecture retenue : **option B** — topologie distribuée « Edge-to-Server » sur le portable d'un apprenant

Sujet de référence : *Mission SENTINEL-X*, Direction Pédagogique Nationale EPSI, session octobre 2026.

Sommaire

1. **1** Contexte, périmètre et variante d'architecture
2. **2** Schéma réseau : sous-réseau, flux et ports
3. **3** Chaîne de collecte : firmware et schéma de câblage
4. **4** Infrastructure et maintien en condition opérationnelle
5. **5** Matrice de sécurité (chiffrement et durcissement)
6. **6** Documentation de l'intelligence artificielle
7. **7** Qualité, tests et reproductibilité
8. **8** Limites connues et pistes d'amélioration

Ce que contient ce dossier

Les chapitres 5 et 6 reprennent les documents sources du dépôt (`docs/SECURITE.md` et `docs/IA.md`) ; les schémas des chapitres 2 et 3 sont produits depuis le code (`infra/docker-compose.yml` , `firmware/sentinel-x/`). Le poster de présentation est livré à part, en A3 portrait (`Workshop2026-M1-G<n>-Poster.pdf`).

1. Contexte, périmètre et variante d'architecture

AetherCorp Industrial Solutions exploite des micro-centrales énergétiques isolées, exposées à trois menaces simultanées : cyberattaques de déstabilisation, intrusions physiques d'espionnage industriel et risques environnementaux (fuite de gaz, surchauffe). Faute de pouvoir maintenir du personnel sur place, l'initiative **SENTINEL-X** confie la surveillance à un boîtier autonome (Edge Node) relié sans fil à un centre de commandement tactique matérialisé par un PC serveur local durci.

Variante d'architecture

Le sujet autorise deux options matérielles. Le consortium retient l'**option B** : le rôle de PC serveur local est tenu par l'ordinateur portable d'un apprenant, qui fait aussi point d'accès Wi-Fi de la table. La webcam USB y est branchée en direct, et l'intelligence artificielle profite de ses capacités de calcul. Le boîtier ne contient que le microcontrôleur et ses composants de captation, et transmet ses flux par Wi-Fi à l'adresse du serveur.

Périmètre livré

- **Edge et IoT** — micrologiciel C++ pour l'ESP8266 unique de table : lecture cadencée des capteurs, affichage OLED, alarme locale, publication MQTTS.
- **Serveur et API** — API REST et WebSocket hébergée sur le PC serveur, centralisant les métriques et l'historique des événements.
- **Supervision** — interface web : courbes environnementales temps réel, statut logique du boîtier, retour visuel de la webcam, panneau de commande des actionneurs.
- **Intelligence artificielle locale** — vision par webcam et maintenance prédictive sur séries temporelles.
- **Infrastructure** — orchestration des conteneurs serveurs et sécurisation des protocoles d'ingestion.
- **Cybersécurité** — chiffrement des flux, durcissement des conteneurs, journal d'audit et contrôles reproductibles.

Périmètre adapté, validé en début de workshop

Trois éléments du sujet ont été adaptés, en accord avec l'encadrement :

- **Boîtier virtuel** — aucun matériel n'étant distribué, le boîtier est simulé (`scripts/virtual_esp.py`) : mêmes topics, même compte MQTTS, mêmes commandes, même alarme locale que le firmware. Le firmware C++ est écrit et compile.
- **Coque non fabriquée** — la modélisation CAO, l'impression 3D et la gravure laser n'ont pas été réalisées.
- **Pentest croisé du jeudi supprimé** — il n'y a donc pas de rapport d'audit post-pentest ; le chapitre 5 fournit l'auto-audit reproductible.

2. Schéma réseau : sous-réseau, flux et ports

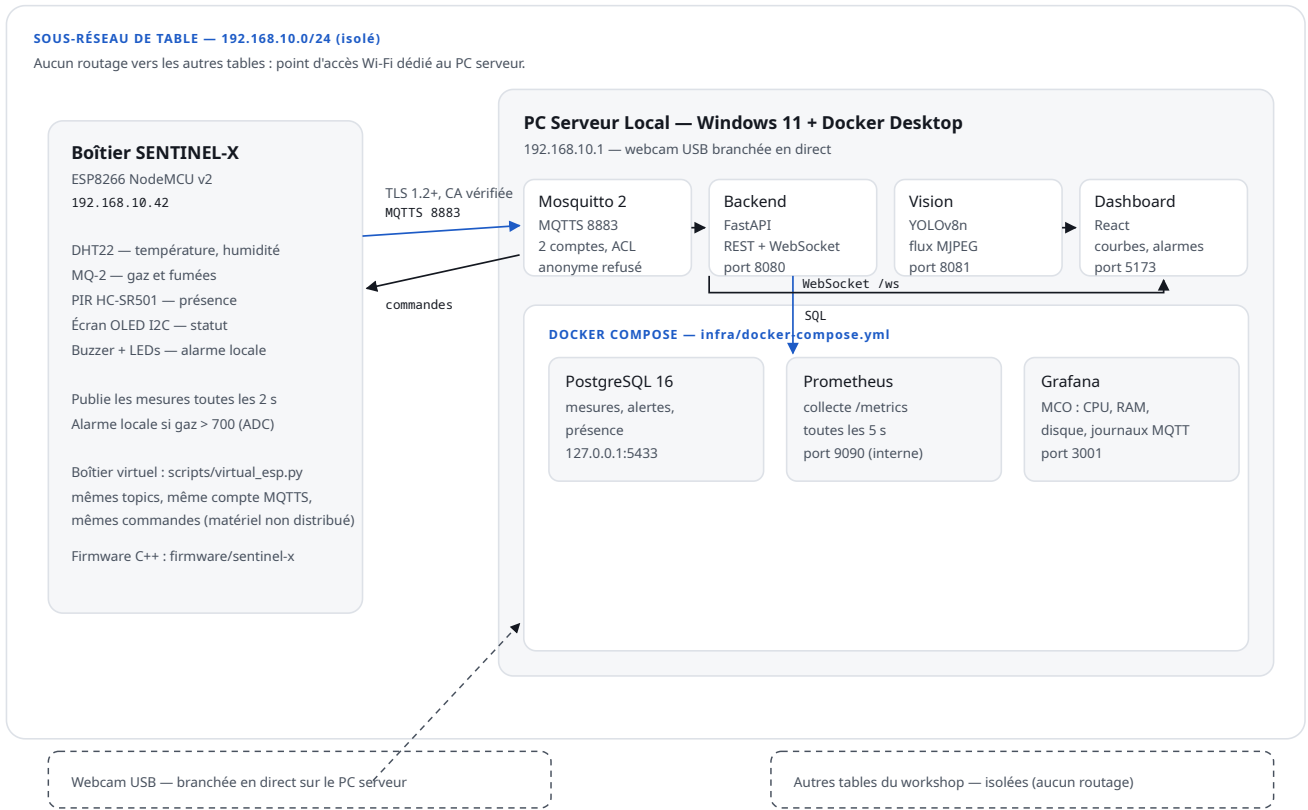


Figure 1 — Topologie de table : le boîtier, le PC serveur local et les flux chiffrés. Les conteneurs Docker sont isolés sur l'hôte.

Plan d'adressage et isolation

La table utilise le sous-réseau 192.168.10.0/24, desservi par le point d'accès Wi-Fi du PC serveur. Aucune route n'est établie vers les autres tables du workshop : les paquets du boîtier ne peuvent atteindre que le PC serveur, et réciproquement. Seuls les services nécessaires sont exposés.

Service	Port	Écoute sur	Protocole
Mosquitto (conteneur)	8883	Réseau de table	MQTTS — TLS 1.2 minimum, compte obligatoire
Backend Fortex	8080	Réseau de table	HTTP (REST) et WebSocket /ws
Flux vidéo (vision)	8081	Réseau de table	MJPEG annoté et /status
Dashboard React	5173	Réseau de table	HTTP (build Vite)
PostgreSQL (conteneur)	5433	localhost uniquement	TCP — injoignable depuis le réseau
Grafana (conteneur)	3001	Réseau de table	HTTP
Prometheus (conteneur)	9090	interne	Collecte de /metrics toutes les 5 s

Le compte MQTT du serveur ne peut pas publier de mesures : l'injection de fausses données est refusée par l'ACL, volontairement. Le port 1883 en clair n'est plus publié.

3. Chaîne de collecte : firmware et schéma de câblage

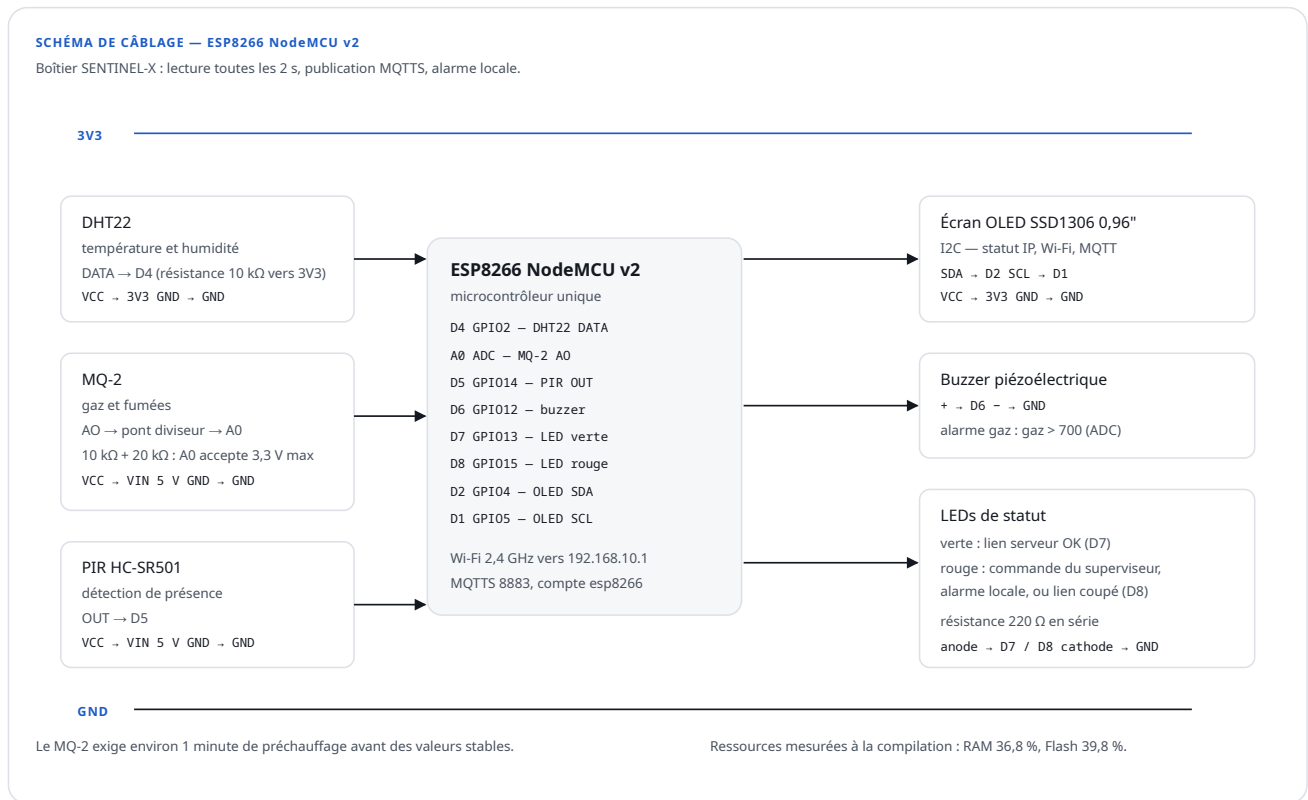


Figure 2 — Câblage du boîtier : un seul microcontrôleur ESP8266, trois capteurs, un écran OLED et les organes d'alerte.

Comportement du firmware

- Lecture du DHT22, du MQ-2 et du PIR **toutes les 2 secondes**.
- Publication en **MQTTS** (TLS, port 8883, compte `esp8266`) sur `sentinel/<id>/sensors : {node_id, temperature, humidity, gas, pir, ip, rssi}`.
- Écoute de `sentinel/<id>/commands : {actuator: buzzer|led, state: true|false}`, déclenchée depuis le dashboard.
- Publication de `online / offline` sur `sentinel/<id>/status` ; le message `offline` est le testament MQTT, publié par le broker si le boîtier disparaît.
- **Garde-fou local** — buzzer et LED rouge si le gaz dépasse 700 (ADC), même sans serveur joignable.

L'entrée A0 du NodeMCU accepte 3,3 V au maximum : la sortie 5 V du MQ-2 passe par un pont diviseur. Le MQ-2 demande environ une minute de préchauffage avant des valeurs stables. Comme l'ESP8266 s'appuie sur l'heure de compilation pour valider le certificat TLS, le module est flashé après la génération des certificats.

4. Infrastructure et maintien en condition opérationnelle

La pile serveur est décrite dans `infra/docker-compose.yml` et se déploie en une commande. Les services sont isolés dans des conteneurs pérennes, redémarrés automatiquement et durcis.

Conteneur	Rôle	Durcissement appliqué
mosquitto	Broker MQTTS des mesures et des commandes	Comptes nominatifs, ACL, anonyme refusé, aucun port en clair
postgres	Mesures, alertes, journal de présence	Publié sur localhost seulement, volume dédié
backend	API REST et WebSocket	<code>read_only</code> , <code>cap_drop</code> , <code>no-new-privileges</code>
prometheus	Collecte des métriques toutes les 5 s	Profil dédié, données sur volume
grafana	Tableaux de bord MCO	Compte PostgreSQL en lecture seule
simulator	Simulateur IoT (profil <code>simulation</code>)	Désactivé par défaut : jamais mélangé aux mesures réelles

Surveillance du MCO

Le backend expose `GET /metrics`, collecté par Prometheus et affiché dans Grafana. Les indicateurs surveillés couvrent à la fois la disponibilité de la chaîne et la santé de la machine hôte, conformément à l'exigence de maintien en condition opérationnelle :

- **Chaîne** — backend en fonctionnement, pont MQTTS connecté, base joignable, fraîcheur de la dernière mesure, dashboards connectés, vision en cours.
- **Volumétrie** — nombre de mesures reçues, messages capteurs rejetés, alertes par type et par source, commandes envoyées, **volume du journal Mosquitto**.
- **Machine hôte** — **CPU**, **RAM** utilisée et **disque**, pour anticiper la saturation face à l'afflux continu des messages.

5. Matrice de sécurité (chiffrement et durcissement)

Consortium FORTEX. Contrôles vérifiés automatiquement par `python scripts/check_security.py`.

1. Chiffrement des flux

Flux	Avant	Après	Preuve
ESP8266 → broker	MQTT en clair, port 1883	MQTTS, TLS 1.2 minimum , port 8883. Le certificat serveur est vérifié par l'ESP8266 (BearSSL, CA FORTEX embarquée)	Écran OLED « MQTT OK (TLS) » ; logs Mosquitto <code>negotiated TLSv1.3</code>
Backend, IA → broker	MQTT en clair	MQTTS , vérification du certificat par la CA FORTEX	<code>check_security.py</code> : <code>TLSv1.3</code> , <code>TLS_AES_256_GCM_SHA384</code>
IA → API	HTTP sans authentification	HTTP + jeton Bearer (secret dans <code>.env</code> , hors Git)	Requête sans jeton : HTTP 401
Backend → PostgreSQL	—	Publié sur localhost uniquement (<code>127.0.0.1:5433</code>), injoignable depuis le réseau	<code>docker compose ps</code> : <code>127.0.0.1:5433->5432/tcp</code>
Grafana → PostgreSQL	—	Compte grafana_ro en lecture seule (SELECT uniquement)	<code>infra/postgres/init/02-grafana-readonly.sh</code>

PKI : `infra/scripts/gen-certs.sh` crée une autorité racine « FORTEX Root CA » et un certificat serveur RSA 2048 valable 2 ans. Il couvre les noms `mosquitto`, `localhost` et l'adresse IP du PC serveur. La clé privée de la CA reste sur le PC serveur ; seul `ca.crt` est distribué.

2. Authentification et droits (moindre privilège)

Compte MQTT	Peut publier	Peut lire	Usage
<code>esp8266</code>	<code>sentinel/+/sensors</code> , <code>sentinel/+/status</code> , <code>fortex/capteurs/mesures</code>	<code>sentinel/+/commands</code>	Boîtier, simulateur
<code>serveur</code>	<code>sentinel/+/commands</code>	<code>sentinel/+/sensors</code> , <code>sentinel/+/status</code> , <code>fortex/capteurs/mesures</code>	Backend, IA
<code>anonyme</code>	refusé	refusé	—

Les mots de passe sont aléatoires (`gen-mqtt-users.sh`) et hachés dans le fichier `passwd` . Un compte volé reste limité : le compte `serveur` **ne peut pas injecter de fausses mesures**, ce qui a été vérifié.

3. Durcissement des conteneurs (infra/docker-compose.yml)

Mesure	Services	Effet
no-new-privileges	tous	Aucune élévation de privilèges dans le conteneur
Utilisateur non-root (UID 10001)	backend	Une faille de l'API ne donne pas root
cap_drop: ALL	backend, simulateur	Aucune capacité Linux
read_only: true + tmpfs /tmp	backend, simulateur	Système de fichiers non modifiable
Rotation des logs (10 Mo × 3)	tous	MCO : le flux MQTT continu ne remplit pas le disque
Healthchecks	postgres, backend	Redémarrage et ordre de démarrage fiables
Limites Mosquitto (max_connections 50, message_size_limit 4096, max_queued_messages 1000)	broker	Atténue un déni de service pendant le pentest croisé

4. Exposition réseau du PC serveur

Port	Service	Exposé
8883	MQTTS	réseau de la table (ESP8266)
8080	API backend	réseau de la table (dashboard)
5173	dashboard	réseau de la table
8081	flux vidéo IA	réseau de la table
5000	contrôle d'accès (biométrie)	localhost uniquement
5433	PostgreSQL	localhost uniquement (127.0.0.1), pour le backend du PC serveur
3001	Grafana (supervision, MCO)	réseau de la table, compte admin obligatoire
1883	MQTT en clair	fermé

Pare-feu Windows, à appliquer en PowerShell administrateur. Ces règles autorisent uniquement le sous-réseau de la table :

```
$table = "192.168.10.0/24"
New-NetFirewallRule -DisplayName "FORTEX MQTTS" -Direction Inbound -Protocol TCP -LocalPort 8883 -RemoteAddress $table -Action Allow
New-NetFirewallRule -DisplayName "FORTEX API" -Direction Inbound -Protocol TCP -LocalPort 8080,5173,8081 -RemoteAddress $table -Action Allow
New-NetFirewallRule -DisplayName "FORTEX bloque MQTT clair" -Direction Inbound -Protocol TCP -LocalPort 1883 -Action Block
```

5. Secrets

- Aucun secret dans Git : les .env, certs/, mqtt-users.env, passwd et firmware/.../sentinel_config.h sont exclus par les .gitignore.
- Les fichiers *.example documentent chaque variable, avec des valeurs factices.

6. Données personnelles (biométrie)

Seules les empreintes faciales sont conservées (128 nombres par agent), jamais les photos : une photo envoyée depuis le dashboard ou la plateforme est convertie en empreinte puis oubliée. Le traitement est 100 % local, et la plateforme n'est accessible que depuis le PC serveur.

La liste est modifiable depuis le dashboard (page « Individus ») ou en ligne de commande (`scripts/enroll.py`) : les deux écrivent dans le même `data/faces.npz`, que la vision `--whitelist` relit à **son démarrage**. La suppression (`DELETE /api/v1/people/{name}`) efface les données et l'empreinte (droit à l'effacement).

7. Application web : OWASP Top 10:2025 (dashboard et API)

Catégorie	Risque identifié	Mesure	Preuve
A01 Contrôle d'accès	Données personnelles (individus, présence) et actions (acquiescement, exercice) ouvertes à tout le réseau	Code opérateur exigé ; présence diffusée par WebSocket aux seuls clients authentifiés	tests/test_backend.py : accès anonyme = 401 ; WebSocket anonyme sans présence
A02 Configuration	Pas d'en-têtes de sécurité ; serveur de développement Vite pendant la démo ; CORS * par défaut	En-têtes nosniff, X-Frame-Options, no-store sur l'API ; CSP stricte (empreintes SHA-256, ni unsafe-inline ni eval) ; démo sur build de production (pnpm run demo) ; aucune origine CORS par défaut	test_security_headers_and_no_wildcard_cors ; 0 violation CSP au navigateur
A03 Chaîne d'approvisionnement	Vite ≤ 6.4.2 : 4 vulnérabilités dont 1 élevée (contournement server.fs.deny sous Windows)	Vite 6.4.4, lockfile pnpm, scripts d'installation limités à esbuild	pnpm audit : aucune vulnérabilité connue
A04 Cryptographie	Dashboard et API en HTTP sur le réseau de la table : le code opérateur circule en clair	— (voir § 8)	—
A05 Injection	Textes saisis (noms, consignes, messages)	React échappe tout rendu ; requêtes SQL paramétrées ; CSP sans script en ligne	revue de code
A06 Conception	Arrêt de la vision sans garde-fou ; photo sans taille maximale	Confirmation explicite et état « Vigilance » tant que la vision est arrêtée ; photo ≤ 8 Mo	test_photo_size_is_bounded

Catégorie	Risque identifié	Mesure	Preuve
A07 Authentification	Code opérateur compilé dans le JavaScript (lisible par tous)	Code saisi par l'opérateur, gardé dans l'onglet (localStorage), jamais dans le bundle	contrôle automatique : code absent des scripts servis
A08 Intégrité	Messages temps réel et disposition locale acceptés sans contrôle	Validation de chaque message WebSocket ; disposition relue et bornée	message malformé ignoré sans plantage
A09 Journalisation	Aucune trace des actions de l'opérateur	data/audit.log : acquittement, commande, exercice, vision, individus, échec d'authentification WebSocket	journal lu après test
A10 Conditions exceptionnelles	Échecs d'action silencieux (console) ; une erreur de composant efface l'écran	Chaque échec est affiché à l'opérateur ; acquittement refusé = alarme rétablie ; un widget en erreur n'affecte pas les autres	tests navigateur

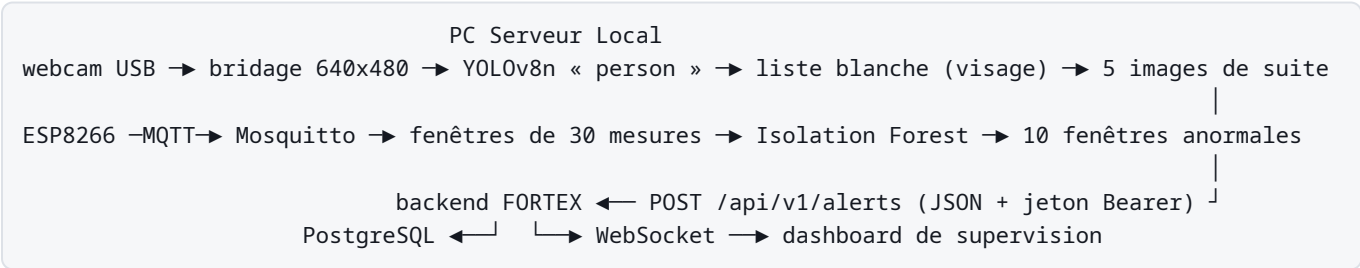
8. Reste à la charge de l'équipe CYBER

- HTTPS pour l'API et le dashboard : reverse proxy TLS, avec la même CA (OWASP A04 : le code opérateur circule aujourd'hui en clair sur le réseau de la table).
- Flux vidéo (:8081/video) servi sans authentification : toute machine du réseau de la table voit la caméra. À protéger (jeton ou passage par l'API).
- Comptes opérateurs nominatifs : le code opérateur est partagé, le journal d'audit ne peut donc attribuer une action qu'à un poste (adresse IP), pas à une personne.
- IA prédictive : aucun signal de vie vers le serveur. Si sentinel_anomaly.py s'arrête, rien ne le signale (le dashboard l'indique « Non supervisé »).
- Accès SSH par clé uniquement (si le serveur est sous Linux ou WSL), et appliquer les règles de pare-feu ci-dessus.
- Pentest croisé du jeudi (Nmap, Wireshark, Metasploit) et rapport d'audit. check_security.py fournit la base de l'auto-audit.

6. Documentation de l'intelligence artificielle

Consortium FORTEX — Workshop EPSI Bac+4, octobre 2026. Dépôt : `Fortexworkshop/ia`.

La brique IA protège la micro-centrale AetherCorp contre deux des trois menaces du cahier des charges : **l'intrusion physique** (vision par ordinateur sur la webcam USB) et **les risques environnementaux** (surchauffe, fuite de gaz) par maintenance prédictive. Tout s'exécute en local sur le PC Serveur Local (option B) : aucune image ni mesure ne quitte la table.



1. Vision intelligente : détection d'intrusion

Script : `scripts/sentinel_vision.py` (module `sentinel/vision.py`).

Étape	Choix	Justification
Capture	Webcam USB branchée sur le PC serveur, OpenCV	Exigé par le sujet
Bridage	Redimensionnement systématique en 640x480	Exigé par le sujet, coût de calcul constant
Modèle	YOLOv8n (Ultralytics), pré-entraîné sur COCO, filtré sur la classe <code>person</code>	Plus petit modèle YOLOv8, temps réel sur CPU
Taille d'inférence	480 px	Meilleur compromis mesuré (voir tableau)
Seuil de confiance	0,5	Écarte les détections douteuses
Anti faux positifs	Personne vue 5 images de suite , puis 10 s entre deux alertes	Un reflet ou une image isolée ne déclenche rien
Liste blanche	Visage reconnu (YuNet + SFace, similarité cosinus $\geq 0,363$) dans la boîte de la personne	Un technicien autorisé n'est pas un intrus
Sorties	Alerte <code>INTRUSION</code> (gravité <code>critical</code>), flux vidéo annoté MJPEG : <code>8081/video</code>	Dashboard et historique

Performance mesurée (exigence : < 100 ms par trame)

Mesure sur la webcam du PC serveur, CPU uniquement, 40 images par taille (`python scripts/bench_vision.py`). Latence = bridage + inférence.

Taille d'inférence	Moyenne	95e centile	Images/s	< 100 ms
640	74 ms	124 ms	13,5	non
480 (retenue)	32 ms	34 ms	31,4	oui
416	26 ms	28 ms	38,2	oui
320	22 ms	23 ms	46,2	oui

La latence de chaque image est affichée en direct sur le flux, en vert sous 100 ms et en rouge au-dessus, pour le prouver pendant la démo.

2. Maintenance prédictive : anomalies cinétiques

Scripts : `scripts/sentinel_train.py` (entraînement), `scripts/sentinel_anomaly.py` (temps réel). Module : `sentinel/anomaly.py`.

Pourquoi pas de seuil statique

Le sujet interdit `if temp > 40`. Un seuil ne voit l'incident qu'une fois qu'il est **déjà** là. L'exemple du sujet, une hausse lente de température corrélée à une micro-déviations du gaz, reste longtemps sous tout seuil raisonnable. On détecte donc **un comportement anormal**, pas une valeur anormale.

Données

Le DHT22 (température, humidité) et le MQ-2 (gaz, valeur analogique 0–1023) sont publiés par l'ESP8266 toutes les 2 s sur `sentinel/<id>/sensors`.

Indicateurs (9 par fenêtre glissante de 30 mesures, soit 1 minute)

Capteur	Indicateurs
Température	moyenne, écart-type, pente (régression linéaire)
Humidité	moyenne, pente
Gaz	moyenne, écart-type, pente
Croisé	corrélation température / gaz

Les pentes et la corrélation captent la dynamique (« cinétique ») : une dérive lente et simultanée des deux capteurs ne ressemble à aucune fenêtre normale.

Modèle

- Normalisation standard (`StandardScaler`), puis **Isolation Forest** (Scikit-Learn, 200 arbres).
- Apprentissage **non supervisé**, sur le régime normal uniquement : il n'est pas nécessaire

d'avoir vécu un incident pour l'entraîner.

- Seuil appris, pas fixé à la main** : une fenêtre est anormale si elle est plus atypique que la plus atypique des fenêtres normales vues à l'entraînement.

- Confirmation** : l'alerte part après **10 fenêtres anormales consécutives** (20 s). La gravité (`warning` ou `critical`) dépend de l'écart au seuil.

- Explicabilité** : l'alerte contient les 9 indicateurs de la fenêtre. Le dashboard peut afficher *pourquoi* l'IA a alerté (par exemple une pente du gaz de +0,63 par mesure et une corrélation température/gaz de 0,77).

Entraînement

1. **Avant le boîtier** : 12 régimes normaux simulés de 600 mesures (`sentinel/simulate.py`), avec niveau de base, oscillation lente, bruit des capteurs et pics isolés. Les fenêtres ne chevauchent jamais deux séries, sinon une coupure créerait une fausse pente apprise comme « normale ».

1. **Avec le boîtier** : enregistrement d'environ 10 minutes en régime normal

(`sentinel_anomaly.py --record`), puis réentraînement sur ces mesures réelles (`sentinel_train.py --csv data/sensors.csv`). Le modèle apprend ainsi le niveau réel de la salle.

Résultats

Évaluation sur des scénarios simulés indépendants de l'entraînement : 50 incidents (surchauffe de +0,04 °C par mesure avec dérive du gaz) et 50 séries normales de 30 minutes. Commande : `python scripts/sentinel_train.py --runs 50`.

Indicateur	Résultat
Incidents détectés	50 / 50
Séries normales avec fausse alerte	1 / 50
Avance sur le seuil critique (40 °C ou gaz 600)	436 s en moyenne, 388 s au minimum

L'IA prévient donc **plus de 6 minutes avant** qu'une alarme à seuil ne sonne.

Validation sur le boîtier virtuel (`sentinel/virtual_box.py`), qui remplace les capteurs physiques, mesurée sur 20 boîtiers avec un bruit de capteur différent à chaque fois :

Cas	Résultat
Fonctionnement normal, 30 minutes par boîtier	0 fausse alerte sur 20
Scénario « surchauffe lente »	20 détections sur 20 , vers 25 °C et un gaz à environ 340, alors que les seuils critiques sont à 40 °C et 600

Le backend conserve un **garde-fou de dernier recours** (`SafetyAlarm`) qui sonne quand le seuil critique est *déjà* atteint. Il est distinct de l'IA et sert à garder le site protégé si le service d'IA s'arrête.

3. Contrôle d'accès du personnel (innovation)

Plateforme : `scripts/sentinel_admin.py` , sur `http://localhost:5000`.

- **Liste blanche** : on enregistre un agent (technicien, garde) à partir de 1 à 10 photos. Seule l'empreinte faciale est conservée (128 nombres par agent), jamais les photos. La vision utilise cette liste pour distinguer un agent autorisé d'un intrus.
- **Registre des présences sur site** : l'agent est reconnu à son visage, puis un geste du pouce (MediaPipe Hand Landmarker, 21 points de la main) enregistre l'heure. Pouce en haut = entrée, pouce sur le côté = pause (aller, puis retour), pouce en bas = sortie. Export CSV possible.
- **RGPD** : ce sont des données biométriques (article 9). Le traitement est 100 % local et chaque agent peut être supprimé de la liste. Pour la démo, seuls des membres de l'équipe volontaires sont enregistrés.

4. Intégration avec les autres briques

Flux	Protocole	Format
ESP8266 → IA, backend	MQTT <code>sentinel/<id>/sensors</code>	<code>{"temperature", "humidity", "gas", "pir"}</code>
IA → backend	POST <code>/api/v1/alerts</code> , jeton Bearer	<code>{node_id, source, type, severity, message, timestamp, data}</code>
Backend → dashboard	WebSocket <code>/ws</code>	<code>reading, alert, command-ack</code>
Vision → dashboard	HTTP MJPEG <code>:8081/video</code>	flux annoté

Le secret (jeton de l'API) reste dans `.env`, exclu de Git.

5. Qualité et reproductibilité

- **62 tests automatisés** (`python -m pytest -q`) : classification des gestes, anti faux

positifs, extraction des indicateurs, détection d'un incident avant le seuil critique, client d'alertes, backend (API, WebSocket, PostgreSQL en mémoire), plateforme web.

- Installation et lancement en une commande : `scripts/setup.ps1, scripts/start.ps1`.
- Toutes les mesures de ce document se reproduisent avec `bench_vision.py` et `sentinel_train.py`.

6. Limites et pistes

Limite	Piste
Pas de capteurs physiques : modèle entraîné et validé sur des données simulées (boîtier virtuel)	Avec du matériel : réentraîner sur les mesures réelles (<code>--record</code>)
YOLOv8n pré-entraîné, non spécialisé (affiche ou écran montrant une personne = détection)	Fine-tuning sur images du site ; zone d'intérêt
Reconnaissance faciale sensible à l'éclairage et à l'angle	3 à 5 photos variées par agent
Flux MQTT et vidéo encore en clair	MQTTS (port 8883) et HTTPS : code déjà prêt, certificats à fournir par CYBER
Vision hors Docker (accès webcam USB sous Windows)	Conteneur avec accès matériel sur Linux / Raspberry Pi

7. Qualité, tests et reproductibilité

Tests automatisés

La suite compte **114 tests** (pytest), exécutables sans matériel :

```
python -m pytest -q                # suite complète
python -m pytest tests/test_backend.py -q # un fichier
```

Contrôles de sécurité et de performance

```
python scripts/check_security.py  # 7 contrôles : TLS, comptes MQTT, ACL, jeton
python scripts/bench_vision.py    # latence de la vision (exigence < 100 ms)
```

Reproductibilité de l'installation

L'installation complète tient en deux commandes, sur le PC serveur Windows : `scripts\setup.ps1 -ServerIp 192.168.10.1` puis `scripts\start.ps1`. `scripts/configure.py` est idempotent : il génère les secrets, la CA TLS, les comptes MQTT, les fichiers `.env` et la configuration du firmware **sans jamais écraser une valeur existante**.

Hygiène du dépôt

- Aucun secret versionné : les `.env`, certificats, mots de passe MQTT et configuration du firmware sont générés localement et exclus par `.gitignore` ; les fichiers `*.example` ne contiennent que des valeurs factices.
- Données personnelles hors du dépôt : empreintes faciales et journaux de présence restent sur le PC serveur.
- Documentation du dépôt en français, commits sémantiques (Conventional Commits).

8. Limites connues et pistes d'amélioration

Ces points sont assumés et documentés : le jury les retrouvera dans le dépôt.

Limite	Impact	Piste
Boîtier physique remplacé par un boîtier virtuel	Aucune preuve matérielle : ni câblage, ni coque imprimée et gravée	Flasher le firmware sur le matériel de table si du matériel est fourni
Firmware jamais exécuté sur un ESP8266 réel	Le fonctionnement sur silicium reste à confirmer	Essai sur banc avec le vrai module, puis comparaison des mesures
API, dashboard et flux vidéo en HTTP sur le réseau de table	Le code opérateur circule en clair ; le sujet demande le chiffrement de bout en bout	Reverse proxy TLS avec la même CA, ou pointer l'origine sur HTTPS
Flux caméra (:8081/video) sans authentification	Toute machine de la table voit la caméra	Jeton ou diffusion du flux via l'API
Durcissement du système d'exploitation non appliqué	Pare-feu et accès SSH restent à régler sur le poste serveur	Règles de pare-feu Windows, accès par clé, preuve conservée
Aucun rapport de pentest croisé	Pas d'audit offensif externe	Poursuivre l'auto-audit (<code>check_security.py</code>) et le rejouer sur la table
Code opérateur partagé, non nominatif	Le journal d'audit identifie un poste, pas une personne	Comptes nominatifs et rôles distincts
IA prédictive sans signal de vie	Un arrêt du script n'est pas alarmé sur le serveur	Exposer sa fraîcheur via <code>/metrics</code>

Réponse à une question probable du jury

Le sujet interdit les structures conditionnelles statiques pour la maintenance prédictive (`if temp > 40`). Le modèle déployé est bien un *Isolation Forest* à seuil appris. Le backend conserve en parallèle un **garde-fou de dernier recours** à seuil fixe, hérité du rôle d'alarme locale : il ne participe pas à la prédiction, ne remplace pas le modèle et reste désactivable. Les deux mécanismes sont distincts et documentés comme tels.